

TOP 10 PYSPARK TRANSFORMATIONS & ACTIONS EVERY DATA ENGINEER SHOULD KNOW



Transformations & Actions

select
filter
groupBy
withColumn
join
orderBy
distinct

collect
count
show
take
first
write
foreach

BY MICHAEL FERNANDES

WHAT ARE TRANSFORMATIONS & ACTIONS?

Transformations

Operations that create a new DataFrame/RDD from an existing one.

- Lazy Execution
- Build a DAG (Directed Acyclic Graph)
- No computation happens immediately

Actions

Operations that trigger execution and return results.

- Start actual computation
- Execute the DAG built by transformations
- Return results or write data to storage

Common Transformations:

- `select()`
- `filter()`
- `withColumn()`
- `join()`
- `groupBy()`
- `orderBy()`

Common Actions:

- `show()`
- `count()`
- `collect()`
- `first()`
- `take()`
- `write()`

SELECT()

Extract only the columns needed for analysis.

- Reduces data movement
- Improves performance
- Makes pipelines cleaner

Example

Selecting customer ID and transaction amount from a billion-row sales dataset.



```
df.select("id", "name", "salary")
```

FILTER() / WHERE()

Keep only rows matching a condition.

Data cleaning

Data validation

Faster downstream processing

Use Case

Filtering active users before generating reports.



```
# filter
```

```
df.filter(df.salary > 50000)
```

```
# where
```

```
df.where("salary > 50000")
```


WITHCOLUMN()

Add derived columns or transform existing ones.

- Feature Engineering
- Data Enrichment
- Business Calculations

Use Case

Creating revenue, profit, tax, or KPI columns.



```
from pyspark.sql.functions import col

df.withColumn(
    "annual_salary",
    col("salary") * 12
)
```

GROUPBY() + AGG()

Summarize large datasets.

- Business Reporting
- KPI Calculation
- Dashboard Metrics

Example

- Average Salary
- Total Revenue
- Customer Count
- Product Sales



```
df.groupby("department") \
    .agg({"salary": "avg"})
```


JOIN()

Merge data from different sources.

Join Types

- Inner Join
- Left Join
- Right Join
- Full Join

Real-World Example

Combining Orders, Customers, and Products tables.



```
orders.join(  
  customers,  
  "customer_id",  
  "inner"  
)
```

DISTINCT() & DROPDUPPLICATES()

Remove duplicate records.

- Better Data Quality
- Accurate Reporting
- Reliable Analytics

Use Case

Removing duplicate customer registrations.



```
# distinct  
df.distinct()
```

```
#dropDuplicates  
df.dropDuplicates(["email"])
```


COUNT()

Count records in a dataset.

- Data Validation
- Record Reconciliation
- Pipeline Monitoring

Example

Verifying source and target row counts after ETL.



```
df.count()
```

SHOW(), COLLECT() & WRITE()

`show()` – Display rows from a DataFrame for inspection.

`collect()` – Retrieve all records from executors to the driver node.

`write()` – Persist DataFrames to storage systems. Save processed data permanently



```
# show
```

```
df.show(5)
```

```
#collect
```

```
data = df.collect()
```

```
# write
```

```
df.write.parquet("/data/output")
```



THANK YOU

BY MICHAEL FERNANDES